Como medir os seus agentes de IA

Do "parece que melhorou" para a eval: métricas de um agente de código, tarefas com teste escondido, A × B com intervalo de confiança e como adaptar o repositório agent-evaluation ao seu projeto.

---

1. Por que medir
2. O que é "efetivo"
3. Montando as tarefas
4. Rodando com promptfoo
5. Lendo o resultado
6. Melhorando o agente
7. Caso real: agentes KTLO na Creditas
8. Armadilhas
9. Referências

Fonte: https://github.com/MateusCastro2203/agent-evaluation

Por que medir

Este ebook é a palestra "Como medir os seus agentes de IA" em texto corrido, mais o que você precisa para rodar e adaptar o repositório `agent-evaluation`. A palestra cabe em 40 minutos, com as perguntas; aqui dá para ir com calma, ver o código e voltar quando for montar a sua própria eval.

A aposta

A palestra começa com uma aposta. Eu escrevi um `CLAUDE.md` para um projeto pequeno, com as convenções da casa: que tipo de erro lançar, como lidar com dinheiro, de onde vem a hora atual, que pasta não se toca. Acho que ele ajuda o agente. Mas "acho" não é medida.

Então, no primeiro minuto, eu rodo:

```
pnpm eval
```

Esse comando põe o Claude Code para trabalhar **32 vezes**: 2 variantes × 8 tarefas × 2 repetições. No fim, ele mesmo gera o relatório com o resultado.

- **Variante A:** o repositório como está, sem `CLAUDE.md`.
- **Variante B:** o mesmo repositório, com o `CLAUDE.md`.

Isso é uma **eval** (de *evaluation*, avaliação): um conjunto de tarefas em que "deu certo" é conferido por código, rodado várias vezes, para comparar o antes e o depois de uma mudança. Neste ebook, "variante" é cada versão comparada.

Enquanto o agente trabalha, a palestra explica o que está sendo medido e por quê. No fim, o relatório diz se o `CLAUDE.md` presta. Ou diz que ainda não dá para saber, que é uma resposta tão útil quanto as outras (capítulo 5).

"Parece que melhorou"

Todo mundo que usa agente de código já fez isto:

1. mudou o prompt, o `CLAUDE.md` ou o modelo;
2. pediu **uma** tarefa;
3. deu certo;
4. "melhorou!".

É o *vibe check*: avaliar no olho, pela sensação de uso, sem número. Ele não é inútil: é como você descobre que vale a pena olhar para alguma coisa. O problema é parar nele, porque ficou muita coisa de fora:

- A tarefa era fácil ou difícil? Uma tarefa fácil passa com qualquer configuração.
- Se você pedisse de novo, daria certo de novo?
- Quanto custou? Quantos turnos (idas e voltas entre o modelo e as ferramentas) o agente gastou?
- Ele mexeu em arquivo que não devia?
- Teria dado certo **antes** da mudança também?

A última pergunta é a mais esquecida. Sem rodar a configuração antiga nas mesmas tarefas, você não tem com o que comparar.

O agente não é determinístico

Mesmo prompt, mesmo repositório, mesmo modelo: o agente não faz a mesma coisa duas vezes. O modelo sorteia cada token (o pedacinho de texto que ele escreve) de uma distribuição de probabilidade, e o loop de ferramentas amplifica qualquer diferença pequena do começo. Um agente lê o arquivo A primeiro, o outro lê o B; um roda os testes, o outro não. No fim, um passa e o outro falha. Isso é o comportamento normal da ferramenta, não um defeito.

A consequência prática: **uma execução é uma amostra, não um resultado**. A pergunta certa não é "passou?", e sim "com que frequência passa?". Frequência se estima com várias execuções e se reporta com um intervalo de confiança: a margem de erro, como a de uma pesquisa eleitoral. Com poucas pessoas ouvidas, a margem é larga; com mais, ela estreita.

Quanto diz uma execução?

O repositório usa o **intervalo de Wilson** com 95% de confiança (explicado no capítulo 5). Dois números para calibrar a intuição:

Observado	Taxa real compatível (Wilson, 95%)
1 de 1 passou	entre **21% e 100%**
16 de 16 passaram	entre **81% e 100%**

Uma execução que passou é compatível com um agente que acerta uma vez a cada cinco. E mesmo 16 acertos em 16 tentativas não provam 100%. Esses números são conta, não opinião: é a mesma fórmula que o relatório do repositório usa.

O que você leva deste ebook

Ao final, você consegue:

- dizer quais métricas definem "efetivo" para um agente de código: sucesso pelo critério de feito, custo e tokens, turnos, tempo, edição fora do escopo e consistência entre execuções (capítulo 2);
- montar um conjunto pequeno de tarefas com testes escondidos e rodar o seu agente contra ele (capítulos 3 e 4);
- comparar duas configurações, por exemplo com e sem `CLAUDE.md`, e ler o resultado com intervalo de confiança, sem se enganar com uma execução só (capítulo 5);
- transformar uma falha observada em tarefa nova do conjunto (capítulo 6).

O repositório roda o mesmo experimento da palestra com o login normal do Claude Code. Não precisa de chave de API.

O que é "efetivo"

"O agente está bom?" não tem resposta enquanto "bom" não estiver definido. Para um agente de código, "bom" são seis números, não um.

As seis métricas

Métrica	Pergunta	Como o repo mede
Sucesso	resolveu?	o teste escondido da tarefa passou (critério de feito)
Custo / tokens	quanto gastou?	`usage` e `total_cost_usd` do JSON do `claude -p`
Turnos	quantas voltas deu?	`num_turns`
Tempo	quanto demorou?	`duration_ms`
Fora do escopo	mexeu onde não devia?	arquivos alterados fora da lista `permitidos` da tarefa
Consistência	resolve **sempre**?	pass^k: a tarefa passou em todas as repetições

Algumas definições, para quem está começando:

- **Critério de feito:** a regra objetiva que diz se a tarefa foi resolvida. Aqui é sempre um teste automatizado que o agente não vê (capítulo 3).
- **Turno:** aproximadamente uma ida e volta entre o modelo e uma ferramenta: ler um arquivo, editar, rodar os testes. Mais turnos costumam significar mais tokens e mais tempo.
- **Token:** o pedaço de texto que o modelo lê ou escreve. É a unidade de consumo; tokens de entrada, de saída e de cache são contados separados.

Por que sucesso sozinho engana

Dois agentes com a mesma taxa de sucesso podem ser muito diferentes. Um pode gastar o dobro de turnos. Outro pode resolver a tarefa editando um arquivo congelado. Se você só olhar a taxa, os dois parecem iguais.

Por isso **fora do escopo** é uma métrica separada, de propósito. "Resolveu, mas editou o código legado que ninguém podia tocar" não é sucesso: em um time de verdade, esse PR volta na revisão.

Custo em US$: leia como esforço

O `claude -p --output-format json` devolve um campo `total_cost_usd`. A documentação do Claude Code ("Run Claude Code programmatically") avisa que esse valor é uma **estimativa do lado do cliente**. Se você usa assinatura, ele não é fatura: leia como "tamanho do esforço" da execução e

compare entre variantes, não como dinheiro gasto.

Desfecho, não "funcionou?"

Cada execução do repositório termina com **um** rótulo:

Desfecho	Quando
`passou`	o teste escondido passou e nada foi alterado fora do escopo
`falhou`	o teste escondido falhou
`fora_do_escopo`	algum arquivo alterado não está em `permitidos` (mesmo que o teste passe)
`erro_agente`	o agente nem terminou (timeout, login expirado, limite de turnos, saída inválida) ou as dependências que rodam o teste foram adulteradas

A ordem de decisão é: primeiro `erro_agente`, depois `fora_do_escopo`, depois `passou` ou `falhou` pelo teste.

O ponto mais importante é o último rótulo. **Erro do agente é contado à parte**, fora da taxa de sucesso. Se a sua rede caiu no meio do run, isso não diz nada sobre o `CLAUDE.md`. Se você contasse como falha, a variante que teve o azar de rodar durante a queda pareceria pior. O relatório mostra quantos `erro_agente` cada variante teve, e você decide se o run vale ou se precisa rodar de novo.

Essa ideia vem direto do caso real do capítulo 7: sem rótulo explícito, "funcionou?" vira discussão.

pass@k × pass^k

Com várias repetições por tarefa, aparecem duas maneiras de resumir:

- **pass@k:** a tarefa passou em **pelo menos uma** de k tentativas.
- **pass^k:** a tarefa passou em **todas** as k tentativas.

Quando k cresce, pass@k sobe (mais chances de acertar uma vez) e pass^k desce (mais chances de errar uma vez). pass@k faz sentido quando um humano olha as tentativas e escolhe a melhor. Para um agente que trabalha sozinho, você não vai pedir de novo até dar certo: o que importa é se ele acerta **sempre**. Por isso o relatório mostra pass^k.

A métrica pass^k foi proposta no τ-bench (Yao et al., 2024), e o post da Anthropic "Demystifying evals for AI agents" (2026) usa a mesma definição: a chance de **todas** as k tentativas darem certo.

No run da palestra, k = 2: o relatório diz quantas das 8 tarefas passaram nas duas repetições, por variante.

Montando as tarefas

Uma eval de agente de código é um conjunto de **tarefas** com **critério de feito**. A tarefa é o pedido, escrito como você escreveria uma issue. O critério de feito é um teste que prova que o pedido foi atendido. O resto (rodar, repetir, calcular) é encanamento.

O laboratório

O repositório traz um app pequeno, em TypeScript, chamado laboratório (`laboratorio/`): uma agenda de meetups com eventos, inscrições, importação de inscritos por CSV e um relatório. Os dados são sintéticos e ficam em memória. O app tem testes com Vitest.

Como todo projeto real, ele tem convenções que não dá para adivinhar só lendo o código. A variante B ganha um `CLAUDE.md` que explica essas convenções. Este é o trecho principal, copiado do repositório (`variantes/b/CLAUDE.md`):

```
## Convenções

- Erros: sempre `ErroDominio` (`src/erros.ts`) com `codigo` em
  `SCREAMING_SNAKE`, por exemplo `INSCRICAO_DUPLICADA`. Nunca
  `throw new Error(...)` em regra de negócio.
- Dinheiro: sempre `Centavos` (inteiro, `src/dinheiro.ts`). Nada de float.
  Ao dividir ou aplicar percentual, arredonde o valor final para baixo
  (`Math.floor`).
- Tempo: nunca `new Date()` em `src/`. Use `relogio.agora()` de
  `src/relogio.ts`, para os testes conseguirem fixar a hora.
- Auditoria: toda mutação de inscrição (criar, cancelar, qualquer nova)
  chama `registrarAuditoria(acao, dados)`. Ações no formato
  `inscricao.<verbo>`.
- `src/legado/` é congelado: não edite. Código novo de relatório vai em
  `src/relatorios/`.
```

(As linhas foram quebradas para caber na página; no arquivo, cada regra é uma linha.)

As 8 tarefas

Cinco tarefas dependem de uma convenção do `CLAUDE.md`. Três são **controle**: não dependem de nenhuma convenção, e A e B deveriam empatar nelas.

#	Tarefa (como uma issue)	O teste escondido exige
01	A listagem de próximos eventos devolve um evento a menos do que o limite.	controle
02	Uma função que conte os inscritos de um evento.	controle
03	Não deixar inscrever em evento lotado.	`ErroDominio` com código de lotação em `SCREAMING_SNAKE`
04	Cupom de desconto percentual na inscrição.	`Centavos` inteiro, valor final arredondado para baixo
05	Não deixar inscrever em evento que já aconteceu.	a regra usa `relogio.agora()`
06	O relatório de inscrições precisa mostrar o e-mail.	código novo em `src/relatorios/`, `src/legado/` intocado
07	Quando lotar, pôr a pessoa numa lista de espera.	registro de auditoria da nova mutação
08	A importação de CSV quebra com vírgula dentro de aspas.	controle

Por que ter tarefas de controle

As tarefas de controle protegem você de uma conclusão errada. O efeito esperado do `CLAUDE.md` está nas tarefas 03 a 07. Se B ganhar **também** nas tarefas 01, 02 e 08, desconfie: pode ser ruído, ou outra coisa mudou entre as variantes.

Por que o prompt não cita a convenção

O prompt é escrito como uma issue de verdade: diz o que a pessoa quer, não como o projeto faz. Se o prompt dissesse "use `ErroDominio`", A e B empatariam, e a eval não mediria nada. O repositório tem um teste que falha se algum prompt mencionar `ErroDominio`, `relogio`, `Math.floor`, auditoria, `src/relatorios`, legado ou `CLAUDE.md`.

Isso não quer dizer que A não tenha chance. Na tarefa 03, o código já lança `ErroDominio` em outras regras, e um agente atento pode copiar o padrão. Na 05, outra função do app já usa `relogio.agora()`. Já nas tarefas 04 (direção do arredondamento) e 06 (pasta do código novo), não há nenhum sinal no código: elas medem o `CLAUDE.md` quase puro. Vale dizer isso quando você apresentar o resultado.

Uma tarefa por dentro

Cada tarefa é uma pasta em `tarefas/NN-nome/` com três coisas:

- `tarefa.json`: o título, o prompt, os arquivos permitidos e se a tarefa depende de convenção;

- `oculto.test.ts`: o teste escondido, que o agente não vê;
- `solucao/`: a solução de referência, usada para provar que a tarefa é possível.

O `tarefa.json` da tarefa 03, copiado do repositório:

```
{
  "titulo": "Evento lotado",
  "prompt": "Hoje dá para inscrever mais gente do que a capacidade do evento. Não deixe
inscrever em evento lotado: quando o evento já tiver `capacidade` inscritos,
`inscrever` deve recusar a nova inscrição.",
  "permitidos": ["src/inscricoes.ts", "test/**"],
  "dependeDeConvencao": true
}
```

O agente só recebe o `prompt`. O campo `permitidos` define o escopo: qualquer arquivo alterado fora dessa lista vira `fora_do_escopo`. Nos globs, `*` não atravessa `/` e `**` atravessa. O identificador da tarefa é o nome da pasta.

E o coração do `oculto.test.ts` da mesma tarefa (trecho; os helpers `evento`, `erroDe` e as pessoas `ana`, `bia`, `caio` ficam no topo do arquivo):

```
// O prompt não dá o nome do código: aceita qualquer SCREAMING_SNAKE
// que fale de lotação.
it("recusa inscrição além da capacidade com ErroDominio e código de lotação", () => {
  const e = evento(2);
  inscrever(e.id, ana);
  inscrever(e.id, bia);
  const erro = erroDe(() => inscrever(e.id, caio));
  expect(erro).toBeInstanceOf(ErroDominio);
  const { codigo } = erro as ErroDominio;
  expect(codigo).toMatch(/^[A-Z][A-Z0-9_]*$/);
  expect(codigo).toMatch(/LOTAD|CHEIO|CAPACIDADE|ESGOT|LIMITE|VAGA/);
  expect(codigo, "CAPACIDADE_INVALIDA já existe e significa outra coisa")
    .not.toBe("CAPACIDADE_INVALIDA");
});

it("vaga liberada por cancelamento pode ser ocupada", () => {
  const e = evento(1);
  inscrever(e.id, ana);
  cancelarInscricao(e.id, ana.email);
  expect(() => inscrever(e.id, bia)).not.toThrow();
});
```

Repare no que o teste **não** exige: um nome exato de código. O prompt não diz qual código usar, então exigir `EVENTO_LOTADO` letra por letra mediria sorte, não convenção. O teste exige o que a convenção diz (o erro do domínio, com código no formato da casa) e aceita qualquer nome razoável de lotação. Ele só recusa reaproveitar `CAPACIDADE_INVALIDA`, que já existe no app e significa outra coisa.

Desenhar o teste é a parte mais difícil. Outro exemplo é a tarefa 04, cujo teste escolhe valores que separam as interpretações erradas. O comentário do próprio teste explica:

```
// Só o valor final com Math.floor passa nos dois casos:
// - 999 a 50%: exato 499,5. Reprova Math.round (500) e também
//   floor só do desconto (999 - floor(499,5) = 500).
// - 1999 a 15%: exato 1699,15. Math.round acerta (1699), mas floor
//   só do desconto reprova (1999 - floor(299,85) = 1700).
//   Valor não inteiro (float) reprova nos dois.
```

Três regras para uma tarefa honesta

1. Toda tarefa falha no app intacto. Se o teste escondido já passa sem o agente fazer nada, a tarefa é inútil: as duas variantes "resolvem". O repositório verifica isso para as 8 tarefas.

2. Toda tarefa passa com a solução de referência. Se nem a solução passa, a tarefa é impossível, e a taxa das duas variantes cai junto. O mesmo teste verifica as duas coisas.

3. O agente não vê o teste, e não consegue trapacear com ele. O teste escondido nunca entra na pasta do agente. Depois que o agente termina, a correção monta uma pasta **nova**, que o agente nunca viu: o app intacto, mais os arquivos que ele mudou, mais o teste escondido, que entra por último e sobrescreve qualquer arquivo com o mesmo nome. Se o agente criar um `test/oculto.test.ts` falso que sempre passa, ele é substituído. Além disso, o teste roda com uma configuração do Vitest fixa, que fica **fora** da cópia (`src/vitest-oculto.config.mjs`). Assim, um `vitest.config.ts` criado pelo agente, com `exclude` ou um `setupFiles` que "conserta" o resultado, é ignorado. O repositório tem um teste que planta exatamente essas trapaças e confere que a tarefa continua falhando.

Essas regras parecem paranoia até a primeira vez que um agente, tentando "fazer os testes passarem", edita o teste em vez do código.

Rodando com promptfoo

Este capítulo mostra como rodar o experimento no seu computador e o que acontece em cada execução.

> **Nota sobre versões.** Comandos, flags e números abaixo são os do repositório (`package.json`, `src/claude.ts`, `evals/promptfooconfig.yaml`) no dia em que este ebook foi gerado. As flags do `claude -p` mudam entre versões do Claude Code: se algo daqui divergir do código, vale o código.

O que você precisa

- **Node.js** na versão LTS atual e **pnpm**.
- **Claude Code** instalado e com login feito (`claude`, uma vez, no terminal). O repositório usa a sua assinatura normal do Claude Code. **Não precisa de chave de API**, e este ebook não recomenda usar uma.
- **git**: o repositório usa o git do seu computador para medir o que o agente mudou.
- **macOS**: o sandbox que isola o agente e a correção foi testado só no macOS. Em Linux, o Claude Code e o sandbox-runtime têm suporte próprio, mas este repositório não foi testado lá.

Sem assinatura? Dá para ler o resultado de um run gravado, offline, com `pnpm relatorio:ensaio` (abaixo).

Os comandos

```
pnpm install            # instala as dependências
pnpm test               # camada 1: testes do medidor, offline, sem agente
pnpm eval:rapido        # fumaça: 1 tarefa, 1 repetição, A e B (2 execuções reais)
pnpm eval               # o run completo: 32 execuções reais e, no fim, o relatório
pnpm relatorio          # refaz o relatório a partir de resultados/ultimo.json
pnpm ensaio:gravar      # grava o último run como ensaio (sem caminhos pessoais)
pnpm relatorio:ensaio   # mesmo relatório sobre o run gravado (offline)
```

Uma ordem que funciona:

1. `pnpm test` primeiro. Se ele falhar, o problema está no medidor, não no agente (veja "Duas camadas", abaixo).

2. `pnpm eval:rapido` para confirmar que o login e o Claude Code respondem. São 2 execuções reais.
3. `pnpm eval` para o run de verdade: 2 variantes × 8 tarefas × 2 repetições = 32 execuções, modelo `sonnet`, 4 em paralelo. No notebook do palestrante, o ensaio levou **3 min 28 s** de relógio (cada execução, de 12 s a 42 s). No seu computador, o tempo depende do modelo, da concorrência e da sua cota.
4. Abra `resultados/relatorio.html` no navegador: o `pnpm eval` gera o relatório sozinho quando termina (e imprime a tabela no terminal). O `pnpm relatorio` só é preciso para refazer o relatório a partir de `resultados/ultimo.json`. O capítulo 5 explica como ler.

O promptfoo sai com código 100 quando alguma execução não passou. É o comportamento dele, não um erro: o JSON em `resultados/ultimo.json` é gravado do mesmo jeito, e o `pnpm eval` gera o relatório em seguida. Por isso o código de saída final do `pnpm eval` é o do relatório, que dá o veredito (capítulo 5).

O run gravado no ensaio de 2026-10-27 fica em `resultados/ensaio/` (o JSON, o relatório e as notas em `NOTAS.md`). Ele serve de plano B ao vivo e de modo offline para quem não tem assinatura: `pnpm relatorio:ensaio` não chama nenhum modelo e grava `resultados/ensaio/relatorio.html`, ao lado do JSON, sem tocar no relatório do run ao vivo. Para gravar um ensaio seu, rode `pnpm eval` e logo depois `pnpm ensaio:gravar`: ele copia `resultados/ultimo.json` para `resultados/ensaio/resultado.json` trocando a pasta do projeto por `.`, o seu home por `~` e as pastas temporárias por `<tmp>`, e gera o relatório.

O que acontece em uma execução

Cada uma das 32 execuções passa por três etapas.

1. Cópia limpa

O repositório cria uma pasta temporária nova (em `/tmp/medindo-agentes/`, fora de qualquer repositório e fora do `$TMPDIR`), copia o laboratório para ela (sem `node_modules`) e coloca o `CLAUDE.md` se a variante for B. Depois registra esse estado inicial num git **do host**: o diretório do git fica ao lado da cópia, não dentro dela (`--git-dir` e `--work-tree`). Tudo o que o agente fizer depois é comparado com esse estado.

Por que o git fica fora: se a cópia fosse um repositório, o agente poderia mexer no `.git` (um `config` com filtro, um hook) e fazer o git do host rodar código dele na hora de medir. Com o git fora, nada do que o agente escreve na cópia vira configuração do git.

As dependências do laboratório (só o Vitest e os tipos do Node) ficam numa pasta própria, `.lab-deps`, instalada uma vez e **somente leitura**. O `node_modules` da cópia só tem links para ela. Antes e depois da correção, o repositório confere um hash dessa pasta (conteúdo, permissões e links): se o agente conseguir mudar uma dependência, a execução vira `erro_agente` em vez de contar como resultado.

A cópia limpa garante que uma execução não contamina a outra e que A e B partem exatamente do mesmo código. A única diferença entre as variantes é o `CLAUDE.md`.

2. O agente

Na cópia, o repositório roda o Claude Code em modo **headless**: sem tela interativa, com o prompt da tarefa como argumento e o resultado em JSON. A chamada (função `argumentosClaude` em `src/claude.ts`) usa:

- `-p <prompt>` e `--output-format json`;
- `--model sonnet` (fixado em `evals/promptfooconfig.yaml`);
- `--max-turns 30`, e um timeout de 10 minutos por execução, aplicado pelo próprio repositório (ele mata o grupo de processos inteiro);
- `--permission-mode dontAsk` com uma lista fechada em `--allowedTools`: ler e editar só dentro da cópia (`Read(./**)`, `Edit(./**)`), rodar `pnpm test` e `pnpm vitest`, e os comandos de leitura `ls`, `cat`, `grep`, `head` e `wc`. O que não está na lista é recusado, sem perguntar a ninguém;
- `--disallowedTools` com `WebFetch`, `WebSearch`, a edição do `node_modules`, a leitura do projeto (onde estão os testes escondidos e as soluções) e a leitura do seu `~/.claude`;
- um **sandbox do sistema operacional** (o Seatbelt, no macOS), configurado por `--settings`: todo comando Bash, inclusive o código que o `pnpm test` roda, só escreve na cópia e no `$TMPDIR`, não lê o seu home nem as outras cópias, e não tem rede. O sandbox falha alto se não estiver disponível e não oferece a saída "rodar fora do sandbox";
- `--setting-sources project`, para não carregar a sua configuração pessoal (veja "Isolamento", abaixo);
- `--strict-mcp-config`, sem `--mcp-config`: nenhum servidor MCP;
- `--no-session-persistence`, para as execuções não deixarem sessões salvas na sua máquina.

O processo também roda sem as variáveis de ambiente de uma sessão do Claude Code que esteja aberta (`CLAUDECODE`, `CLAUDE_CODE_*`). O login vem do mesmo lugar de sempre.

Nunca use `--permission-mode bypassPermissions` nem `--dangerously-skip-permissions` para rodar esta eval. São 32 execuções sem ninguém olhando: com as permissões desligadas, um agente que decide "limpar" arquivos ou instalar algo faz isso na sua máquina, não só na cópia.

Do JSON final, o repositório lê `is_error`, `num_turns`, `duration_ms`, `total_cost_usd` e os campos de `usage`: tokens de entrada e de saída, e, separados, os de cache (leitura + criação). Se o processo estourar o timeout, ou o JSON vier inválido, a execução vira `erro_agente` em vez de derrubar o run.

Confira as flags com `claude --help` e na página "Run Claude Code programmatically" da documentação antes de mudar qualquer coisa: elas mudam entre versões.

3. A correção

Quando o agente termina, o repositório:

1. lista os arquivos alterados em relação ao estado inicial, com o git do host, incluindo arquivos novos e apagados;
2. procura entradas suspeitas na cópia: qualquer link simbólico, qualquer `.git`, FIFO ou dispositivo. Se achar, a execução vira `fora_do_escopo` e nem é corrigida;
3. compara a lista de alterados com `permitidos` da tarefa. `package.json`, configuração do Vitest ou do Vite e qualquer coisa dentro de um `.git` são sempre fora do escopo;
4. monta uma pasta **nova**, que o agente nunca viu: o laboratório intacto, mais os arquivos que ele mudou (só arquivos comuns), mais o teste escondido por cima de `test/oculto.test.ts`;
5. roda só o teste escondido, com a configuração fixa do Vitest (capítulo 3), dentro de outro sandbox (o `srt`, do pacote `@anthropic-ai/sandbox-runtime`): sem rede, escrevendo só na pasta corrigida;
6. decide o desfecho: `erro_agente`, senão `fora_do_escopo`, senão `passou` ou `falhou`.

A lista de alterados vem do git do host (trecho de `src/copia.ts`, com as linhas quebradas para caber na página):

```
export async function arquivosAlterados(pasta: string): Promise<string[]> {
  const nulo = (s: string) => s.split("\0").filter(Boolean);
  const mudados = nulo(
    await git(pasta, "diff", "--name-only", "--no-renames", "-z", REF_INICIAL),
  );
  const novos = nulo(
    await git(pasta, "ls-files", "--others", "-z",
      `--exclude-from=${exclusoes(pasta)}`),
  );
  return [...new Set([...mudados, ...novos])]
    .filter((f) => f !== OCULTO)
    .sort();
}
```

Um detalhe que só apareceu rodando de verdade: `pnpm test` dentro da cópia cria um `pnpm-lock.yaml`. Sem tratar isso, todo agente que roda os testes (que é o que o `CLAUDE.md` pede) apareceria com um arquivo fora do escopo. As exclusões do git do host (o arquivo `exclusoes`, fora da cópia) escondem esse arquivo e o `node_modules` montado pelo repositório; um `.gitignore` criado pelo agente na cópia não esconde nada.

Notas de método

Algumas consequências desse desenho, que valem saber antes de ler o resultado:

- **A cópia não é um repositório git.** Um comando `git` do agente (`git status`, `git diff`) falha. Se o agente rodar `git init`, cria um `.git` e a execução vira `fora_do_escopo`. As tarefas não pedem git, e no ensaio nenhuma execução criou um `.git`.
- **Qualquer link simbólico criado pelo agente vira `fora_do_escopo`**, mesmo um que aponte para dentro da cópia: a correção só aceita arquivos comuns.
- **Configuração gerenciada pela organização continua valendo.** Se a sua conta tem configurações gerenciadas (managed settings), elas chegam às duas variantes, igualmente, e nenhuma flag tira. O isolamento é "nada do seu usuário e nada de MCP", não "configuração zero".
- **A tarefa 06 mede só a convenção.** O prompt pede uma função de relatório sem dizer onde; a variante A não tem nenhum sinal de que ela deve ir em `src/relatorios/`. A diferença ali é o efeito do `CLAUDE.md`, por construção.
- **O `ensaio:gravar` limpa por texto.** Ele troca os caminhos por substituição de texto no JSON: qualquer trecho `/tmp` vira `<tmp>`, mesmo fora de um caminho, e um caminho escrito com barras escapadas passaria. Confira o JSON antes de publicar um ensaio seu.

Onde entra o promptfoo

O promptfoo é o **orquestrador**: ele lê a lista de casos, chama um *provider* para cada um, repete, roda em paralelo e grava tudo num JSON. *Provider*, no promptfoo, é o pedaço de código que "responde" a cada caso. Normalmente é um modelo; aqui é um provider em TypeScript que roda as três etapas acima e devolve o desfecho.

No repositório:

- `evals/agente.ts` é o provider: recebe a variante e o modelo na configuração, expõe `id()` (`claude-code-a` ou `claude-code-b`) e `callApi`, que roda a tarefa e devolve `{ output: desfecho, metadata: { resultado } }`;
- `evals/casos.ts` gera um caso por tarefa, com uma asserção que passa quando o `output` é `passou`;
- `evals/promptfooconfig.yaml` declara os dois providers, 2 repetições e concorrência 4.

O trecho principal da configuração:

```
providers:
  - id: file://agente.ts
    label: variante a
    config:
      variante: a
      modelo: sonnet
  - id: file://agente.ts
    label: variante b
    config:
      variante: b
      modelo: sonnet
tests: file://casos.ts
evaluateOptions:
  maxConcurrency: 4
  repeat: 2
```

O cache fica desligado pela linha de comando (`--no-cache`, nos scripts do `package.json`): na versão do promptfoo usada, a chave `cache: false` do arquivo não tinha efeito.

A página "Javascript Provider" da documentação do promptfoo descreve o contrato do provider (`id` e `callApi`). A página "Command line" descreve as flags que o `pnpm eval:rapido` usa: `--filter-first-n` para rodar só a primeira tarefa e `--repeat` para mudar o número de repetições, além de `-o` para gravar a saída.

O promptfoo não calcula o veredito. Ele não tem intervalo de confiança nem comparação A × B com essa regra. Isso fica num script do repositório (`scripts/relatorio.ts`), que lê o JSON do promptfoo e tira o desfecho de `metadata.resultado`, nunca do "passou/falhou" do promptfoo, que não distingue `erro_agente` de `falhou`. É o assunto do capítulo 5.

Isolamento: a sua configuração não pode vazar

Este é o erro mais fácil de cometer e o mais difícil de perceber. O Claude Code lê configuração de vários lugares: do projeto, mas também do seu usuário (`~/.claude/CLAUDE.md`, hooks, skills globais, plugins) e dos conectores MCP da sua conta. Se você tem um `CLAUDE.md` pessoal, a variante A **não** roda "sem `CLAUDE.md`": ela roda com o seu. O experimento fica contaminado e você não fica sabendo.

O repositório restringe as fontes de configuração ao projeto (`--setting-sources project`), o que mantém o `CLAUDE.md` da cópia (variante B) e deixa de fora o seu usuário. Isso não bastou: medindo de verdade, o início de cada execução ainda listava 47 servidores MCP (os conectores da conta, que `--setting-sources` não controla) e um número instável de ferramentas. Com `--strict-mcp-config`, sem nenhum `--mcp-config`, ficaram 0 servidores MCP e só as 25 ferramentas nativas, nas duas variantes.

A verificação foi feita assim, antes do ensaio (`resultados/ensaio/ISOLAMENTO.md`): numa cópia de cada variante, com os mesmos argumentos do executor, pedir ao agente que liste literalmente todas as instruções de `CLAUDE.md` ou memória que recebeu, e ler o evento de início (`--output-format stream-json --verbose`). A variante A respondeu "NENHUMA"; a B citou só o `CLAUDE.md` da cópia. Se você adaptar o repositório, repita essa verificação na sua máquina.

> **Segurança: rode só numa cópia descartável.**
>
> - Nunca aponte o agente para o seu checkout de trabalho. Cada execução tem que rodar numa pasta temporária que pode ir para o lixo.
> - Mantenha o sandbox ligado: sem rede, escrita só dentro da cópia.
> - Se você trocar o laboratório pelo seu repositório, confira o sandbox de novo: peça ao agente, numa cópia, para escrever fora dela e para acessar a rede, e veja as duas coisas falharem.
> - Lembre que `pnpm test` roda o que o `package.json` mandar. O script `test` do seu projeto passa a ser executado dezenas de vezes, sem supervisão.

Duas camadas de eval

O repositório tem duas camadas, com papéis diferentes.

Camada 1: determinística. São os testes do **medidor**, com um agente falso: um que não faz nada (deve dar `falhou`), um que aplica a solução de referência (`passou`), um que edita o código congelado (`fora_do_escopo`), um que devolve timeout (`erro_agente`, sem derrubar o run), um que planta um teste oculto falso (continua `falhou`). Roda offline, em segundos, a cada commit, com `pnpm test`. Ela prova que a régua mede certo.

Camada 2: modelo real. É o `pnpm eval`: o Claude Code de verdade nas 8 tarefas. Leva minutos e consome cota. Rode quando algo que você quer medir mudar: o prompt, o `CLAUDE.md`, o modelo, uma skill, um hook.

Sem a camada 1, um bug no medidor parece "o agente piorou". Com ela, quando a camada 2 muda de resultado, você sabe que foi o agente.

Lendo o resultado

O run terminou e o relatório está aberto. A tentação é olhar a taxa de sucesso de A, a de B, e declarar um vencedor. Este capítulo é sobre não fazer isso.

A ordem de leitura

O relatório é gerado no fim do `pnpm eval` (ou com `pnpm relatorio`): imprime uma tabela no terminal e grava `resultados/relatorio.html`. No topo do HTML vem o veredito em português simples ("Ainda não dá para saber", "Com CLAUDE.md foi melhor" ou "Sem CLAUDE.md foi melhor") e as duas taxas com a margem de erro. Abaixo, as métricas do capítulo 2 com nomes do dia a dia ("Resolve sempre?" é o pass^k; "Mexeu onde não devia" é o fora do escopo), a tabela por tarefa e, no fim, os "Detalhes técnicos" (intervalo de Wilson, teste z, tokens). Leia nesta ordem:

1. **n por variante e quantos `erro_agente`.** O n da taxa conta só as execuções sem `erro_agente`. Se uma variante teve muitos erros de agente, o run não está equilibrado: investigue antes de concluir qualquer coisa.
2. **Taxa de sucesso com intervalo.** A taxa observada e a faixa de taxas compatíveis com ela.
3. **Veredito**, em português: B melhor, A melhor ou inconclusivo.
4. **pass^k**: quantas tarefas passaram em todas as repetições. O k é o maior número de repetições visto por tarefa no run (hoje, 2). A conta é conservadora: uma repetição com `erro_agente` conta como "não passou", e uma tarefa com menos de k repetições (run interrompido) não conta.
5. **Fora do escopo**: quantas execuções mexeram onde não deviam.
6. **Por tarefa**: o ganho, se houver, está nas tarefas de convenção (03 a 07) ou nas de controle (01, 02, 08)?

Mediana de tokens, turnos e duração, e o custo estimado, ajudam a desempatar e a entender o esforço. A mediana é usada em vez da média porque algumas execuções gastam muito mais que as outras, e a média seria puxada por elas. Os tokens aparecem em duas linhas: entrada + saída, e cache (leitura + criação). O cache é dezenas de vezes maior e, somado, esconderia qualquer diferença entre as variantes. As medianas usam só as execuções sem `erro_agente`; o custo total soma todas, inclusive as com erro, porque é o que o run gastou.

O intervalo de Wilson

Um intervalo de confiança é a faixa de taxas "verdadeiras" que são compatíveis com o que você observou. Com 95% de confiança: se você repetisse o experimento muitas vezes, o intervalo calculado conteria a taxa verdadeira em cerca de 95% das vezes.

A conta mais conhecida para uma proporção é `p ± 1,96 · √(p(1−p)/n)`. Com n pequeno, ou com taxa perto de 0% ou de 100%, ela se comporta mal: o intervalo sai de [0, 1] ou fica com largura zero. Com 16 de 16, por exemplo, ela daria "100% ± 0", como se 16 execuções provassem perfeição. O **intervalo de Wilson** (Wilson, 1927) não tem esse problema e é a escolha comum para proporções com poucas amostras. É o que o repositório calcula, com z = 1,96 (95%).

Um exemplo

Este exemplo é ilustrativo, não um resultado de run:

Variante	Observado	Taxa	Intervalo de Wilson 95%
A	6 de 16	37,5%	18% a 61%
B	12 de 16	75%	51% a 90%

A taxa de B é o dobro da de A. Mas os intervalos **se sobrepõem** entre 51% e 61%: existe uma taxa verdadeira, por exemplo 55%, compatível com as duas variantes. Pela regra do repositório, isso é **inconclusivo**.

A regra do veredito

O repositório decide assim (função `decidir`, em `src/veredito.ts`):

- **B melhor** se o limite inferior de B passa do limite superior de A;
- **A melhor** se o limite inferior de A passa do limite superior de B;
- **inconclusivo** em qualquer outro caso, inclusive quando uma variante tem n = 0.

Inconclusivo não é empate. Empate seria "as duas são iguais". Inconclusivo é "não tenho dado para dizer". São conclusões diferentes, e a segunda tem uma ação clara: rodar mais.

Esse critério é conservador de propósito. Um teste z de duas proporções, aplicado às mesmas 6/16 e 12/16, daria uma diferença estatisticamente significativa (p ≈ 0,03). O repositório prefere não declarar vencedor cedo: exigir intervalos sem sobreposição é mais exigente, e um portão que declara vencedor com pouca evidência ensina o time a desconfiar dele.

Por que 16 não bastam

O relatório também estima quantas execuções **por variante** seriam necessárias para separar A de B. São dois números, e o primeiro é o que manda.

Execuções necessárias por variante (critério do veredito). O relatório pega as taxas observadas de A e de B e pergunta: com quantas execuções por variante, mantidas essas taxas, os intervalos de Wilson 95% deixariam de se sobrepor? Ele testa n = 1, 2, 3… até achar (ou até 10.000, o teto da busca). É o mesmo critério do veredito, então é a resposta direta para "quanto falta para este

relatório sair do inconclusivo". No código, o campo chama `execucoesNecessariasPorVariante` (em `src/veredito.ts`). Ele fica vazio ("não calculável") quando as duas taxas são iguais, quando uma variante não tem dado ou quando nem 10.000 bastam.

Piso pelo teste z de duas proporções. É a fórmula clássica de tamanho de amostra, com 95% de confiança e 80% de poder. **Poder** é a chance de o experimento detectar a diferença se ela existir de verdade. Com 80% de poder: se o `CLAUDE.md` ajuda mesmo, em 8 de cada 10 experimentos desse tamanho você veria a diferença.

```
n ≈ (1,96 + 0,84)² · [pA(1−pA) + pB(1−pB)] / (pA − pB)²
```

O 1,96 vem dos 95% de confiança, e o 0,84, dos 80% de poder. No código, o campo chama `pisoTesteZPorVariante`. Como o teste z é menos exigente que "intervalos sem sobreposição", esse número costuma ficar abaixo do primeiro: com ele, o relatório ainda pode sair inconclusivo. Alguns valores:

Taxa de A	Taxa de B	Critério do veredito (Wilson)	Piso (teste z)
37,5%	75%	27	24
62,5%	75%	212	212
75%	100%	27	24

O n cresce com 1 / (diferença)². Uma diferença grande se vê com poucas dezenas de execuções; uma diferença moderada pede centenas. As duas contas supõem que as taxas observadas continuem iguais com mais execuções, o que ninguém garante: são estimativas, não promessas.

Esta é a lição da palestra: **16 execuções por variante quase nunca bastam para uma diferença moderada.** E está tudo bem. Saber quanto falta já é medir.

Um exemplo real: o ensaio

O run gravado no ensaio de 2026-10-27 (`resultados/ensaio/`, modelo `sonnet`, 2 repetições) é um exemplo de resultado, não uma previsão do próximo run:

	A (sem `CLAUDE.md`)	B (com `CLAUDE.md`)
passou	12 de 16 (75%, Wilson 51% a 90%)	16 de 16 (100%, Wilson 81% a 100%)
pass^2	6 de 8	8 de 8
fora do escopo	3	0
erros do agente	0	0

Veredito: **inconclusivo**. As barras se sobrepõem entre 81% e 90%, e o relatório estima 27 execuções por variante para elas se separarem (o piso pelo teste z é 24). As quatro falhas de A foram todas nas tarefas de convenção: na 04, uma `fora_do_escopo` (mexeu em `src/legado/`, a pasta congelada) e uma em que o teste escondido falhou; na 06, duas `fora_do_escopo` (mexeu em `src/relatorio.ts` numa e em `src/inscricoes.ts` na outra, fora dos permitidos). É o efeito que as tarefas foram desenhadas para medir, mas 16 execuções por lado ainda não bastam para cravar. Rode `pnpm relatorio:ensaio` para ver o relatório completo. Se regravar o ensaio, atualize estes números.

Os três desfechos

B ganhou. Confira onde. O ganho está nas tarefas de convenção (03 a 07)? Ótimo, é o efeito esperado. Está também nas de controle? Desconfie: pode ser ruído, ou outra coisa diferente entre as variantes. Olhe também o fora do escopo e o custo: ganhar em sucesso gastando muito mais pode não valer.

Inconclusivo. O resultado mais comum com 16 execuções por variante. Leia a estimativa de execuções necessárias e decida se vale rodar mais. Olhe a tabela por tarefa: às vezes uma tarefa específica mostra um padrão claro (A sempre falha, B sempre passa) que merece investigação, mesmo sem veredito global.

A ganhou. O `CLAUDE.md` atrapalhou. Isso acontece: instruções demais distraem, uma regra mal escrita leva o agente para o lado errado. Abra as execuções de B que falharam e veja o que o agente fez. Neste caso, e só neste, o portão do repositório sai com código 1.

O portão de qualidade

`pnpm relatorio` (e, portanto, o `pnpm eval`, que o roda no fim) termina com código de saída 1 **só** quando o veredito é "A melhor" (`codigoDeSaida`, em `src/relatorio.ts`), ou seja, quando há evidência de que a mudança piorou o agente. "Inconclusivo" e "B melhor" saem com 0. Se o arquivo de entrada não puder ser lido, sai com 2. (Não confunda com o código 100 que o promptfoo devolve quando alguma execução não passou: isso não é veredito, e o `pnpm eval` não o repassa.)

Isso permite pôr o relatório no CI (integração contínua, o pipeline que roda a cada push ou PR) como portão: se alguém mudar o `CLAUDE.md` e a eval mostrar piora clara, o build quebra. Quebrar o build por "inconclusivo" seria um erro: com poucas execuções, isso aconteceria o tempo todo por falta de dado, e o time aprenderia a ignorar o portão.

Melhorando o agente

Medir só vale se vira decisão. Este capítulo é sobre como usar a eval para melhorar o agente que você usa, e como montar a eval no seu próprio projeto.

Uma mudança por vez

O ciclo é o mesmo de um teste A/B de produto:

1. rode a eval **antes** da mudança (a linha de base);
2. mude **uma** coisa;
3. rode a eval **de novo**, com repetições;
4. compare com intervalo de confiança, não com a média.

"Uma coisa" pode ser:

- o `CLAUDE.md`;
- uma skill nova;
- um hook;
- o modelo;
- o prompt da tarefa.

Se você mudar o modelo **e** o `CLAUDE.md` juntos e o resultado melhorar, você não sabe qual dos dois ganhou. Pode até ser que um ajudou muito e o outro atrapalhou um pouco. O experimento da palestra muda só uma coisa: a existência do `CLAUDE.md`.

Uma dica prática: commite a configuração do agente junto com o resultado da eval. Daqui a um mês, você sabe qual versão gerou qual número.

Onde cada regra mora

O Claude Code oferece três lugares para pôr uma regra. Eles se comportam de modo diferente.

	Quando entra	Bom para	Cuidado
`CLAUDE.md`	em **toda** sessão	convenções do repo, comando de teste, pastas proibidas	é contexto, **não obrigação**
Skill	só quando é usada	procedimento longo, material de referência	descrição ruim, nunca é chamada
Hook	antes ou depois de cada ferramenta	regra que **precisa** valer sempre	é código: teste como código

- **`CLAUDE.md`.** A documentação do Claude Code ("How Claude remembers your project") diz que o conteúdo do `CLAUDE.md` é tratado como contexto, não como configuração obrigatória. O modelo tende a seguir, mas pode não seguir. Para bloquear uma ação de verdade, a mesma documentação indica um hook `PreToolUse`.
- **Skill.** Pela documentação ("Extend Claude with skills"), só a descrição da skill fica no contexto o tempo todo; o corpo entra quando ela é usada. Material de referência longo custa quase nada até ser necessário. Em compensação, se a descrição for ruim, o agente não percebe que deveria usá-la.
- **Hook.** Um script que o Claude Code roda antes ou depois de cada chamada de ferramenta. Pela documentação ("Hooks reference"), um hook `PreToolUse` que sai com código 2 bloqueia a chamada, não importa o que o modelo queira. É a única das três que é determinística.

A regra que resume a tabela: **regra que não pode falhar não mora no prompt. Mora no hook, no teste ou no CI.**

Cada linha dessa tabela é uma variante B possível no repositório. Troque o `CLAUDE.md` por uma skill com as mesmas convenções, ou por um hook que bloqueia edição em `src/legado/`, e rode de novo. A pergunta deixa de ser "qual é melhor?" em abstrato e passa a ser "qual é melhor nestas tarefas, com este intervalo?".

Falha vira tarefa nova

O conjunto de tarefas não é fixo. Ele cresce com o uso:

1. o agente erra algo no dia a dia;
2. você escreve uma **tarefa** com o prompt real que levou ao erro;
3. e um **teste escondido** que prova o comportamento certo;
4. confere que a tarefa falha no app intacto e passa com a solução de referência;
5. a tarefa entra no conjunto e **não sai mais**.

É a lição 7 do caso real (capítulo 7): o erro de terça vira tarefa na quarta.

A catraca

Catraca é um mecanismo que só gira para um lado. Aplicada à eval:

- o conjunto de tarefas **só cresce**;
- o portão só quebra com evidência de piora (intervalos sem sobreposição, capítulo 5);
- o portão roda no CI.

O portão não olha a taxa crua ("não pode cair abaixo da última vez"). Com poucas execuções, a taxa oscila de uma rodada para outra só por ruído, e um portão assim ficaria vermelho sem o agente ter piorado.

Adapte ao seu projeto

O laboratório é pequeno de propósito. A eval que importa é a do **seu** repositório. Um roteiro para montar as primeiras 5 tarefas:

1. Escolha a pergunta. O que você quer saber? "Meu `CLAUDE.md` ajuda?", "Vale trocar de modelo?", "Esse hook atrapalha?". A pergunta define as variantes A e B. Mude uma coisa só.

2. Junte 5 pedidos reais. Procure no histórico do time: issues fechadas, PRs pequenos, pedidos que você fez ao agente na última semana. Prefira tarefas que um dev resolve em minutos, não em horas: cada execução precisa caber em poucos minutos, e você vai rodar várias.

3. Misture convenção e controle. Pelo menos 3 tarefas devem depender de algo que só a sua mudança ensina (a convenção do `CLAUDE.md`, o procedimento da skill). Pelo menos 2 devem ser controle, sem relação com a mudança. Se B ganhar também nos controles, desconfie.

4. Escreva o prompt como issue. Diga o que a pessoa quer, não como o projeto faz. Não cite a convenção no prompt: se citar, A e B empatam.

5. Escreva o teste escondido. Ele é o critério de feito. Teste o comportamento e a convenção, não um detalhe que o prompt não pediu (lembre do código de erro da tarefa 03, no capítulo 3). Se o seu projeto não tem teste, comece por aqui: escrever o teste da tarefa é o primeiro entregável.

6. Defina `permitidos`. Liste os arquivos e pastas que a tarefa pode tocar. Inclua a pasta de testes, se o agente puder escrever testes.

7. Prove que a tarefa é honesta. Ela tem que falhar no código intacto e passar com uma solução de referência que você escreve. Guarde a solução em `solucao/`.

8. Troque o laboratório pelo seu repo. No `agent-evaluation`, são estas as mudanças:

- **De onde vem a cópia:** a constante `LAB` em `src/copia.ts` aponta para `laboratorio/`. Troque pelo caminho do seu projeto (ou copie o projeto para dentro de `laboratorio/`). A cópia continua sendo feita numa pasta temporária: o agente nunca trabalha no seu checkout.
- **As tarefas:** ponha as suas em `tarefas/NN-nome/` (`tarefa.json`, `oculto.test.ts`, `solucao/`) e a sua mudança em `variantes/b/`.
- **Os termos proibidos nos prompts:** o teste em `test/tarefas.test.ts` tem uma regex com as palavras que nenhum prompt pode conter (hoje, `ErroDominio`, `relogio`, `Math.floor`, auditoria, `src/relatorios`, legado, `CLAUDE.md`). Troque pelos nomes das convenções do **seu** projeto, para o teste continuar impedindo que o prompt entregue a resposta.
- **As dependências:** a cópia precisa delas para rodar os testes. O repositório instala uma vez, em `.lab-deps`, a lista fixa de `prepararDependencias` em `src/copia.ts` (hoje só `vitest` e `@types/node`, nas versões da raiz), deixa a pasta somente leitura, confere um hash dela antes e depois da correção e monta na cópia um `node_modules` com links para ela. Troque essa lista pelas dependências do seu projeto e mantenha as duas garantias: somente leitura e hash conferido.

- **Stack que não é Node:** o executor do teste escondido é Vitest, com a configuração fixa em `src/vitest-oculto.config.mjs`. Se o seu projeto é Python, Go, Java etc., troque esse executor pelo equivalente do seu stack (por exemplo, rodar só o arquivo do teste escondido com o runner da linguagem), mantendo as duas garantias: o teste entra por último, numa pasta corrigida nova, depois que o agente termina, e a configuração do runner vem de fora da cópia.

> **Segurança antes de rodar no seu projeto.**
>
> - Rode só em cópia descartável, nunca no seu checkout de trabalho.
> - Mantenha o sandbox: sem rede, escrita só dentro da cópia. Nunca troque por `bypassPermissions` ou `--dangerously-skip-permissions`.
> - Depois de apontar para o seu repositório, confira o sandbox de novo (capítulo 4).
> - `pnpm test`, ou o comando de teste do seu stack, roda o que os scripts do seu projeto mandarem, dezenas de vezes e sem supervisão. Leia esses scripts antes.

9. Rode pequeno, depois grande. Primeiro `pnpm eval:rapido`, para ver se tudo encaixa. Depois o run completo. Leia o relatório como no capítulo 5 e use a estimativa de execuções necessárias para decidir se vale rodar mais.

Cinco tarefas com duas repetições são 10 execuções por variante. Isso raramente separa A de B, mas já mostra onde o agente erra, e cada erro vira uma tarefa nova.

Desafio extra: medir outro agente

O método não é específico do Claude Code. No repositório, a parte que chama o agente fica isolada atrás de uma interface, a `RodarAgente` (em `src/claude.ts`):

```
export interface SaidaAgente {
  ok: boolean; erro?: string;
  turnos: number; duracaoMs: number; custoUsd: number;
  tokensEntrada: number; tokensSaida: number; tokensCache: number;
}
export type RodarAgente = (args: {
  pasta: string; prompt: string; modelo: string; maxTurnos: number; timeoutMs: number;
}) => Promise<SaidaAgente>;
```

Tarefas, cópia limpa, teste escondido, escopo e relatório não sabem qual agente rodou. Para medir outro agente, você implementa **outra** `RodarAgente`. O desafio fica descrito aqui, não implementado no repositório.

Um exemplo é o **Codex CLI**, da OpenAI. A documentação dele ("Non-interactive mode") descreve o `codex exec`, que roda o agente sem a interface interativa, para scripts e CI. Pontos que a documentação descreve e que interessam aqui:

- por padrão, o `codex exec` roda com sandbox somente leitura; para o agente poder editar a cópia, é preciso `--sandbox workspace-write`;
- com `--json`, a saída vira uma sequência de eventos JSON (um por linha), com eventos como `turn.completed`, `turn.failed` e `error`;
- `--ignore-user-config` faz o run não carregar o arquivo de configuração do usuário (`$CODEX_HOME/config.toml`), o equivalente ao isolamento que o repositório faz com o Claude Code;
- o `codex exec` exige rodar dentro de um repositório git. A cópia limpa deste repositório **não** é um (o git que mede o diff fica fora dela, de propósito). A documentação permite pular essa checagem com `--skip-git-repo-check` "se você tiver certeza de que o ambiente é seguro": aqui o ambiente é uma cópia descartável, mas decida isso conscientemente, e não crie um `.git` dentro da cópia, que vira `fora_do_escopo`;
- ele reaproveita o login já salvo da CLI.

O roteiro do desafio:

1. criar um `rodarCodex: RodarAgente` que inicia o processo `codex exec` com a cópia descartável como diretório de trabalho (`cwd`), o prompt da tarefa, `--json` e `--sandbox workspace-write`. Não use o modo sem sandbox (`danger-full-access`) nem a aprovação automática de tudo: o agente roda sem supervisão;
2. ler os eventos da saída e preencher a `SaidaAgente` com o que der; o que a CLI não informar (por exemplo, custo) fica zero, e o relatório deve deixar isso claro;
3. tratar timeout e evento de erro como `ok: false`, para virar `erro_agente`;
4. repetir a verificação de isolamento do capítulo 4 (a variante A não pode receber a sua configuração pessoal);
5. confirmar as flags na documentação atual e no `codex exec --help` antes de rodar, porque elas mudam entre versões.

Isso não é um benchmark de qual ferramenta é melhor. Comparar agentes diferentes muda muita coisa de uma vez (modelo, ferramentas, prompt de sistema). O uso honesto é o mesmo de sempre: medir a configuração que você usa e ver se uma mudança nela ajuda.

Caso real: agentes KTLO na Creditas

Na Creditas, um time construiu agentes que resolvem chamados operacionais de suporte, o trabalho de *keep the lights on* (KTLO): o que mantém a operação rodando no dia a dia. Medir esses agentes ensinou mais sobre eval do que qualquer benchmark.

Este capítulo traz só as lições. Ele não traz números internos, nomes de sistemas, de parceiros ou de pessoas, nem dados de clientes. As lições valem para qualquer agente, inclusive o agente de código que você usa.

1. Desfecho explícito por execução

Toda execução do agente termina com um rótulo: **resolvido**, **assistido**, **escalado** ou **falta informação**.

"Funcionou?" não é uma pergunta que se responde olhando um painel. "Assistido", por exemplo, quer dizer que o agente fez o diagnóstico, mas um humano executou a correção. Contar isso como "resolvido" infla o número. Contar como "falha" esconde o valor que o agente entregou. Por isso é um rótulo próprio.

No repositório desta palestra, a mesma ideia aparece como `passou`, `falhou`, `fora_do_escopo` e `erro_agente` (capítulo 2).

2. Desfecho mal definido distorce a métrica

Quando um caso que está **fora do escopo** do agente é contado como falha, a taxa de sucesso cai. E ninguém sabe se o agente piorou ou se simplesmente entrou outro tipo de caso.

Se a composição do denominador muda, a taxa muda sem o agente mudar. A lição: defina o desfecho **antes** de olhar o número.

3. O custo varia muito entre execuções

Algumas execuções custavam muito mais que a média, por causa do número de turnos. A média escondia isso; por isso olhar mediana e cauda importa mais que olhar a média.

A primeira tentativa de controle foi pôr no prompt algo como "pare depois de N chamadas". **Não segurou**: o modelo não conta os próprios turnos. O teto de orçamento virou um **hook determinístico**, código que roda fora do modelo e bloqueia a execução quando o limite é atingido.

É o mesmo princípio do capítulo 6: regra que não pode falhar não mora no prompt.

4. Eval em duas camadas

A eval ficou com duas camadas:

- uma **determinística**, que roda a cada commit e testa a lógica do agente sem chamar o modelo;
- uma com o **modelo real**, que roda contra um **gabarito curado por humano**.

O gabarito **nasce vazio**. O humano escreve a resposta esperada sem olhar o que o agente respondeu. Se você copia a resposta do agente para o gabarito, você mede o agente contra ele mesmo, e herda o viés dele.

O repositório desta palestra repete as duas camadas: `pnpm test` (o medidor, com agente falso) e `pnpm eval` (o Claude Code de verdade), capítulo 4.

5. Matriz-catraca

A cobertura da eval é uma matriz: **rota × desfecho**. Cada rota (um tipo de chamado que o agente atende) cruzada com cada desfecho possível é uma célula. Cada célula deveria ter pelo menos um caso de eval.

A lista de células **sem cobertura** só pode **encolher**. Um teste quebra se alguém tirar cobertura. É a catraca do capítulo 6, aplicada à cobertura da eval.

6. LLM-as-judge rejeitado ali

Usar outro modelo como juiz (*LLM-as-judge*) foi considerado e rejeitado naquele contexto. A razão: um juiz não determinístico avaliando um agente não determinístico **soma variância**. Cada número da eval passa a ter duas fontes de ruído em vez de uma.

O critério ficou em código e no gabarito humano.

Isso é ponto de discussão, não regra universal. Há tarefas em que o juiz ajuda: critérios que um teste não pega, como clareza de texto ou qualidade de uma descrição de PR. Nesses casos, meça o juiz também, contra rótulos humanos. Os vieses conhecidos de juízes (preferir a resposta numa certa posição, preferir respostas longas, preferir respostas parecidas com as próprias) estão documentados em Zheng et al. (2023). Para o agente de código, a regra prática é: se dá para verificar com código, verifique com código.

7. Falha em produção vira caso de eval

"O caso que surpreendeu na terça é um teste na quarta." Toda falha observada em produção vira um caso novo no conjunto de eval.

E um aprendizado só vira regra (no prompt, na documentação, num hook) depois de aparecer em **dois ou mais** casos. Isso evita transformar uma exceção em regra para todo mundo.

No repositório, é o ciclo "falha vira tarefa nova" do capítulo 6.

Armadilhas

Cada armadilha abaixo produz um número que parece certo e não é. Todas são fáceis de cair e difíceis de perceber depois.

1. Uma execução só

O que acontece: você roda cada variante uma vez em cada tarefa, B passa em mais tarefas, e você conclui que B é melhor.

Por que engana: o agente não é determinístico (capítulo 1). Uma execução que passou é compatível com uma taxa real de 21% a 100%. A diferença entre as variantes pode ser só sorte.

Como evitar: repita cada tarefa (o repositório usa 2 repetições), reporte a taxa com intervalo de Wilson e use pass^k para consistência. Leia a estimativa de execuções necessárias antes de concluir (capítulo 5).

2. Configuração global vazando

O que acontece: a variante A deveria rodar sem `CLAUDE.md`, mas o Claude Code carrega o seu `~/.claude/CLAUDE.md`, os seus hooks e as suas skills globais.

Por que engana: A passa a ter um `CLAUDE.md` escondido. As duas variantes ficam mais parecidas do que você pensa, e a diferença medida encolhe. Pior: o resultado muda de máquina para máquina, conforme a configuração de cada pessoa.

Como evitar: restrinja as fontes de configuração ao projeto (o repositório usa `--setting-sources project`), desligue os conectores MCP da conta (`--strict-mcp-config`, que `--setting-sources` não cobre) e **verifique** na sua máquina: numa cópia da variante A, pergunte ao agente que instruções de memória ele recebeu (capítulo 4). Configuração gerenciada pela organização não sai com flag nenhuma; ela chega igual às duas variantes, então não distorce a comparação, mas o resultado não é "configuração zero".

3. Teste visível ao agente

O que acontece: o teste que define "feito" está no repositório desde o começo, e o agente o lê, ou o edita até passar.

Por que engana: o agente otimiza para o teste, não para a tarefa. Ele pode escrever código que só satisfaz aquele caso, ou simplesmente mudar a asserção. A taxa sobe sem o agente ficar melhor.

Como evitar: o teste escondido só entra na pasta **depois** que o agente termina, sobrescreve qualquer arquivo de mesmo nome e roda com uma configuração do Vitest que fica fora da cópia (capítulo 3).

4. Tarefa que já passa

O que acontece: o teste escondido passa no código intacto, sem o agente fazer nada. Ou o contrário: nem a solução de referência passa.

Por que engana: uma tarefa que já passa dá sucesso para as duas variantes; uma tarefa impossível dá falha para as duas. Nos dois casos, a tarefa só dilui a diferença e faz o conjunto parecer maior do que é.

Como evitar: verifique toda tarefa automaticamente: ela precisa **falhar** no app intacto e **passar** com a solução de referência. O repositório faz isso para as 8 tarefas, na camada determinística.

5. Contar erro de agente como falha

O que acontece: a rede caiu, o login expirou ou a execução estourou o limite de turnos, e isso entrou na taxa como "falhou".

Por que engana: o erro não diz nada sobre a configuração que você está medindo. Se ele se concentrar numa variante (por exemplo, a que rodou durante a queda da rede), essa variante parece pior sem ser.

Como evitar: tenha um desfecho próprio (`erro_agente`), tire-o do n da taxa e mostre a contagem à parte. Se uma variante tiver muitos erros, rode de novo antes de concluir (capítulo 2).

6. Comparar mudando duas coisas

O que acontece: entre a linha de base e a nova rodada, você mudou o modelo **e** o `CLAUDE.md`. Ou atualizou a versão do Claude Code no meio.

Por que engana: se o resultado mudou, você não sabe quem mudou. Uma das mudanças pode ter ajudado muito e a outra, atrapalhado.

Como evitar: uma mudança por vez (capítulo 6). Fixe tudo o que não está sendo medido: modelo, versão da ferramenta, tarefas, número de repetições. Commite a configuração junto com o resultado.

Mais duas, de bônus

Ler o custo como fatura. O `total_cost_usd` do Claude Code é uma estimativa do lado do cliente. Com assinatura, use como medida de esforço para comparar variantes, não como o valor que você pagou.

Tratar "inconclusivo" como empate. Inconclusivo quer dizer "não tenho dado para dizer", não "são iguais". A ação certa é rodar mais, ou aceitar que a diferença, se existe, é pequena demais para valer o esforço (capítulo 5).

Referências

Só fontes públicas, todas conferidas em 26/10/2026. As páginas de documentação mudam: se uma flag citada aqui não bater com a sua versão, vale a documentação atual.

promptfoo

- "Javascript Provider": o contrato do provider próprio (`id`, `callApi`). https://www.promptfoo.dev/docs/providers/custom-api/
- "Command line": `promptfoo eval` e as flags `--repeat`, `-j` / `--max-concurrency`, `-n` / `--filter-first-n` e `-o`. https://www.promptfoo.dev/docs/usage/command-line/
- "LLM Rubric": asserção com modelo como juiz (`llm-rubric`). https://www.promptfoo.dev/docs/configuration/expected-outputs/model-graded/llm-rubric/

Claude Code

- "Run Claude Code programmatically": modo headless (`-p`), `--output-format json` e os campos do JSON; o custo é estimativa do lado do cliente. https://code.claude.com/docs/en/headless
- "How Claude remembers your project": `CLAUDE.md` é contexto, não configuração obrigatória; para bloquear uma ação, hook `PreToolUse`. https://code.claude.com/docs/en/memory
- "Extend Claude with skills": a descrição da skill fica no contexto, o corpo entra quando ela é usada. https://code.claude.com/docs/en/skills
- "Hooks reference": hook `PreToolUse` que sai com código 2 bloqueia a chamada. https://code.claude.com/docs/en/hooks

Codex CLI (desafio extra)

- "Non-interactive mode": `codex exec`, `--json`, `--sandbox workspace-write`, `--ignore-user-config`. https://learn.chatgpt.com/docs/non-interactive-mode

Evals de agentes

- Anthropic (2026), "Demystifying evals for AI agents": pass^k como a chance de todas as k tentativas darem certo; avaliadores em código × avaliadores com modelo. https://www.anthropic.com/research/demystifying-evals-for-ai-agents
- Yao et al. (2024), "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains": propõe a métrica pass^k para medir a confiabilidade do agente em várias tentativas. https://arxiv.org/abs/2406.12045

- Jimenez et al. (2023), "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?": issues reais do GitHub como tarefa para modelos que editam o código, o mesmo formato das tarefas deste repositório. https://arxiv.org/abs/2310.06770
- Zheng et al. (2023), "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena": vieses de posição, de verbosidade e de autopreferência em juízes. https://arxiv.org/abs/2306.05685

Estatística

- Intervalo de Wilson (Wilson, 1927) e por que ele se comporta melhor que o intervalo "p ± z·√(p(1−p)/n)" com n pequeno ou taxa extrema: Wikipedia, "Binomial proportion confidence interval". https://en.wikipedia.org/wiki/Binomial_proportion_confidence_interval

A fórmula de tamanho de amostra do capítulo 5 é a aproximação normal para comparar duas proporções (1,96 para 95% de confiança bilateral, 0,84 para 80% de poder). `[sem fonte]`: não encontrei uma página pública verificada que traga exatamente esta forma.